TP RAGOps & Métrologie

Évaluer votre RAG avec Ragas

Durée : 3
Objectif : Apprendre à mesurer la qualité de votre RAG juridique

---

CE QUE VOUS ALLEZ FAIRE

1. **Générer automatiquement des questions de test** (30 min)
2. **Évaluer votre RAG avec Ragas** (60 min) ← Le cœur du TP
3. **Ajouter une évaluation par LLM** (45 min)
4. **Tracer les performances** (30 min)

PARTIE 1 : Questions de Test

Le Problème

Vous avez un RAG. Comment l'évaluer ?

Il vous faut des **questions de test** avec leurs **réponses attendues**.

❌ **À la main :** 1 heure pour 10 questions (ennuyeux)
✅ **Avec un LLM :** 10 minutes pour 15 questions (automatique)

Ce que vous allez faire

Créer un générateur automatique avec Evol-Instruct :

Document
↓
Extraire des infos clés (facts)
↓
Générer des questions SIMPLES
↓
ÉVOLUTION : Rendre certaines questions plus complexes ← Evol-Instruct

↓
Générer les réponses attendues

Evol-Instruct = Faire évoluer les questions du simple au complexe

Exemple :

Info : "M. A... au CHU de Martinique"

Question SIMPLE (easy) :
→ "Où travaille M. A... ?"

Question ÉVOLUÉE (medium) :
→ "Pourquoi M. A... travaille-t-il au CHU de Martinique ?"

Question COMPLEXE (hard) :
→ "Quelles sont les conséquences juridiques du lieu de travail de M. A... ?"

But : Avoir des questions variées (faciles, moyennes, difficiles)

ÉTAPE 1 : Créer la classe (15 min)

Fichier : `legal_rag/evaluation.py`

Classes à créer :

```
@dataclass
class GoldenSample:
    """Une question de test"""
    query: str              # La question
    ground_truth: str       # La réponse attendue
    metadata: dict = None   # Infos (difficulté, source)

class SyntheticDatasetGenerator:
    """Génère des questions automatiquement avec Evol-Instruct"""

    def extract_facts(self, document_text):
        """Extraire 5 infos importantes du document"""
        # TODO: Appeler Mistral pour extraire

    def generate_question(self, fact):
        """Créer une question SIMPLE à partir d'une info"""
        # TODO: Appeler Mistral pour générer
```

```
    def evolve_question(self, simple_question):
        """ÉVOLUTION : Rendre la question plus complexe
(Evol-Instruct)"""
        # TODO: Appeler Mistral pour complexifier

    def generate_answer(self, document_text, question):
        """Créer la réponse attendue"""
        # TODO: Appeler Mistral pour répondre
```

Prompts à utiliser :

```
# Pour extract_facts :
"""
Extrait 5 informations clés de ce document juridique.

Document: {text}

Format JSON: {"facts": ["info 1", "info 2", ...]}
"""

# Pour generate_question :
"""
Crée une question SIMPLE à partir de cette information.

Information: {fact}

Question simple:
"""

# Pour evolve_question (ÉVOLUTION - Evol-Instruct) :
"""
Rends cette question plus complexe en ajoutant du raisonnement.

Question simple: {question}

Question complexe (nécessite analyse et compréhension):
"""

# Pour generate_answer :
"""
Réponds à cette question selon le document (2-3 phrases max).

Document: {text}
Question: {question}
```

```
Réponse:
"""
```

ÉTAPE 2 : Générer 15 questions

Stratégie Evol-Instruct :

- 8 questions SIMPLES (easy)
- 5 questions ÉVOLUÉES (medium)
- 2 questions COMPLEXES (hard)

Script à créer :

```
# Charger 2-3 documents
documents = ["doc1.xml", "doc2.xml"]

# Générer
generator = SyntheticDatasetGenerator()
samples = []

for doc in documents:
    facts = generator.extract_facts(doc)

    for i, fact in enumerate(facts):
        # 1. Question simple (easy)
        simple_q = generator.generate_question(fact)
        answer = generator.generate_answer(doc, simple_q)
        samples.append(GoldenSample(
            query=simple_q,
            ground_truth=answer,
            metadata={"difficulty": "easy"}
        ))

        # 2. ÉVOLUTION : Question complexe (medium/hard)
        if i % 2 == 0:  # 1 question sur 2
            complex_q = generator.evolve_question(simple_q)
            answer = generator.generate_answer(doc, complex_q)
            samples.append(GoldenSample(
                query=complex_q,
                ground_truth=answer,
                metadata={"difficulty": "medium"}
            ))
```

```
# Sauvegarder
with open("golden_dataset.json", "w") as f:
    json.dump([s.to_dict() for s in samples], f)
```

Explication :

- On génère d'abord une question simple
- Puis on la fait évoluer (1 fois sur 2) avec `evolve_question()`
- Ça crée de la variété : 8 easy + 7 medium/hard ≈ 15 questions

---

ÉTAPE 3 : Vérifier Checkpoint :

- Fichier `golden_dataset.json` créé
- 15 questions générées
- Variété de difficultés (easy, medium, hard)
- Questions ÉVOLUÉES différentes des simples

Regardez 2-3 questions :

- Les questions simples sont-elles claires ?
- Les questions évoluées sont-elles plus complexes ?

Exemple de bonne évolution :

Simple : "Où travaille M. A... ?"
Évoluée : "Pourquoi le lieu de travail de M. A... est-il juridiquement pertinent ?"

PARTIE 2 : Évaluer avec Ragas

Qu'est-ce que Ragas ?

Ragas = Un outil qui note automatiquement votre RAG sur 4 critères.

Analogie simple :

Vous êtes un étudiant qui passe un examen.
Ragas est le prof qui corrige et donne une note sur 4 critères :

1. Est-ce que tu as trouvé les bons documents ? (Retriever - Qualité)
2. Est-ce que tu as TOUS les documents nécessaires ? (Retriever - Couverture)
3. Est-ce que tu as inventé des choses ? (Generator - Hallucinations)
4. Est-ce que ta réponse est claire ? (Generator - Pertinence)

Les 4 Notes de Ragas

📊 Note 1 : context_precision

Question : Les documents trouvés sont-ils pertinents ?

Exemple :

Question: "Quel est le dispositif ?"

Documents trouvés par votre RAG:
✅ Doc 1: "Le dispositif est Cassation"
✅ Doc 2: "Date de l'arrêt: 6 novembre 2025"
❌ Doc 3: "M. A... habite en Martinique"

Note: 2/3 = 0.67 (67%)

Si la note est basse (<60%) :

- Votre RAG trouve trop de documents inutiles
- **Action :** Améliorer le tri des résultats

📊 Note 2 : context_recall

Question : Avez-vous trouvé TOUS les documents nécessaires ?

Exemple :

Réponse attendue: "Le dispositif est Cassation. Dossier 475420."

Documents trouvés:
✅ Doc 1: "Dispositif: Cassation" → Info trouvée
❌ Doc avec "Dossier 475420" → Info manquante

Note: 1/2 = 0.50 (50%)

Si la note est basse (<70%) :

- Votre RAG manque des documents importants
- **Action :** Chercher dans plus de documents (augmenter K)

📊 Note 3 : faithfulness

Question : Avez-vous inventé des informations ?

Exemple :

Documents trouvés: "M. A... au CHU de Martinique"

Votre réponse: "M. A... travaille à Paris"

Note: 0/1 = 0.00 (0%) → HALLUCINATION !

Si la note est basse (<70%) :

- Votre RAG invente des choses
- **Action :** Améliorer le prompt du générateur

📊 Note 4 : answer_relevancy

Question : Votre réponse est-elle précise et claire ?

Exemple :

Question: "Quel est le dispositif ?"

Réponse verbeuse:
"Le dispositif de l'arrêt est Cassation. L'arrêt a été rendu le 6 novembre 2025 par la cour. M. A... était représenté par un avocat..."

Note: 0.60 (60%) → Trop long, pas assez ciblé

Si la note est basse (<70%) :

- Vos réponses sont trop longues ou hors-sujet
- **Action :** Demander des réponses plus courtes

ÉTAPE 4 : Installer Ragas (5 min)

pip install ragas pandas datasets

ÉTAPE 5 : Préparer les données (20 min)

Ce qu'il faut faire :

Pour chaque question de test :

1. Lancer votre RAG (recherche + génération)
2. Récupérer les documents trouvés
3. Récupérer la réponse générée
4. Mettre tout ça dans un tableau

```
Code :
from datasets import Dataset

# Tableau vide
data = {
    'question': [],
    'contexts': [],       # Les documents trouvés
    'answer': [],         # La réponse générée
    'ground_truth': []    # La réponse attendue
}

# Pour chaque question
for sample in golden_dataset:
    # 1. Recherche
    results = pipeline.search(sample.query, n_results=5)

    # 2. Génération
    answer = generator.generate_answer(sample.query, results)

    # 3. Ajout au tableau
    data['question'].append(sample.query)
    data['contexts'].append(results['documents'][0])
    data['answer'].append(answer)
    data['ground_truth'].append(sample.ground_truth)

# Conversion pour Ragas
dataset = Dataset.from_dict(data)
```

Aide : `results['documents'][0]` est une liste de textes

ÉTAPE 6 : Lancer l'évaluation (20 min)

```
from ragas import evaluate
from ragas.metrics import (
    context_precision,
    context_recall,
    faithfulness,
    answer_relevancy
)

# Lancer l'évaluation
result = evaluate(
    dataset,
    metrics=[
        context_precision,
        context_recall,
        faithfulness,
        answer_relevancy
    ]
)

# Afficher les résultats
print(f"Qualité Retriever    : {result['context_precision']:.2f}")
print(f"Couverture Retriever : {result['context_recall']:.2f}")
print(f"Pas d'hallucination  : {result['faithfulness']:.2f}")
print(f"Pertinence réponse   : {result['answer_relevancy']:.2f}")
```

ÉTAPE 7 : Interpréter vos résultats

Exemple de résultats :

Qualité Retriever : 0.45
Couverture Retriever : 0.52
Pas d'hallucination : 0.87
Pertinence réponse : 0.70

Diagnostic :

1. **Quelle est votre note la plus basse ?**
    - Ici : context_precision (0.45)
2. **Qu'est-ce que ça veut dire ?**

    - Votre Retriever trouve beaucoup de documents inutiles
3. **Que faire ?**

    - Améliorer le tri des résultats
    - Ajouter un re-ranking
    - Utiliser un meilleur modèle d'embedding

Remplissez ce tableau :

Note	Votre Score	Bon ou Mauvais ?	Action
context_precision	_____	_____	_____
context_recall	_____	_____	_____
faithfulness	_____	_____	_____
answer_relevancy	_____	_____	_____

Barème :

- 0.80 : Excellent ✅

- 0.60-0.80 : Correct ⚠️
- < 0.60 : À améliorer ❌

PARTIE 3 : LLM Judge

Le Principe

Un LLM "juge" qui note la qualité de vos réponses sur 3 critères :

1. **Fidélité :** Basée sur les documents ?
2. **Clarté :** Bien écrite ?
3. **Complétude :** Toutes les infos ?

ÉTAPE 8 : Créer le juge

```
class LLMJudge:
```

```
    def judge_answer(self, query, answer, contexts):
        """Note une réponse sur 3 critères (0-10)"""

        prompt = f"""
Note cette réponse sur 3 critères (0-10):

1. FIDÉLITÉ: Basée sur les documents ?
2. CLARTÉ: Bien écrite ?
3. COMPLÉTUDE: Toutes les infos ?

DOCUMENTS: {contexts}
QUESTION: {query}
RÉPONSE: {answer}

Format JSON:
{{
  "fidelite": <0-10>,
  "clarte": <0-10>,
  "completude": <0-10>,
  "note_globale": <moyenne>,
  "explication": "..."
}}
"""

        # Appeler Mistral
        response = mistral.chat.complete(
            model="mistral-large-latest",
            messages=[{"role": "user", "content": prompt}],
            response_format={"type": "json_object"}
        )

        return json.loads(response.choices[0].message.content)
```

ÉTAPE 9 : Tester (15 min)

```
judge = LLMJudge()

# Prendre une question
sample = golden_dataset[0]
results = pipeline.search(sample.query)
answer = generator.generate_answer(sample.query, results)

# Noter
```

```
scores = judge.judge_answer(
    sample.query,
    answer,
    results['documents'][0]
)

print(f"Fidélité     : {scores['fidelite']}/10")
print(f"Clarté       : {scores['clarte']}/10")
print(f"Complétude   : {scores['completude']}/10")
print(f"Note globale: {scores['note_globale']}/10")
```

PARTIE 4 : Observabilité avec LangSmith

Le Principe

LangSmith = Un outil professionnel pour observer votre RAG en temps réel.

Ce qu'il fait :

- Trace toutes les étapes (recherche, génération)
- Mesure les temps d'exécution
- Affiche une interface web pour visualiser
- Détecte les erreurs

Analogie : C'est comme les DevTools de Chrome, mais pour votre RAG.

Interface LangSmith :

```
┌──────────────────────────────────────────────────────────┐
│ Trace : RAG Query                                        │
│ Durée totale : 4.2s                                      │
├──────────────────────────────────────────────────────────┤
│ ✅ Retrieval      0.3s  [Détails →]                      │
│ ❌ Re-ranking     3.1s  [Détails →]                      │ ← Goulot !
│ ✅ Generation     0.8s  [Détails →]                      │
└──────────────────────────────────────────────────────────┘
```

ÉTAPE 10 : Installer et configurer LangSmith 1. Installation :

pip install langsmith langchain

2. Créer un compte (gratuit) :

- Allez sur : https://smith.langchain.com/
- Créez un compte
- Notez votre clé API

3. Configuration :

```
import os

# Votre clé API LangSmith
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "votre_cle_ici"
os.environ["LANGCHAIN_PROJECT"] = "TP-RAGOps"  # Nom de votre projet
```

ÉTAPE 11 : Instrumenter votre RAG

Principe : Envelopper vos fonctions avec `@traceable`

```
from langsmith import traceable

@traceable(name="Retrieval")
def search_documents(query, n_results=5):
    """Recherche dans ChromaDB"""
    results = pipeline.search(query, n_results=n_results)
    return results

@traceable(name="Generation")
def generate_answer(query, contexts):
    """Génération avec Mistral"""
    answer = generator.generate_answer(query, contexts)
    return answer

@traceable(name="RAG_Query")
def full_rag_query(query):
    """Pipeline RAG complet"""
    # 1. Recherche
    results = search_documents(query)

    # 2. Génération
    answer = generate_answer(query, results['documents'][0])
```

```
    return answer
```

Ce que fait `@traceable` :

- Enregistre automatiquement le début/fin de chaque fonction
- Mesure le temps d'exécution
- Capture les paramètres (query, n_results...)
- Envoie tout à LangSmith

ÉTAPE 12 : Tester et visualiser

1. Lancez une requête :

```
# Une seule requête pour tester
answer = full_rag_query("Quel est le dispositif ?")
print(answer)
```

2. Ouvrez LangSmith :

- Allez sur : https://smith.langchain.com/
- Cliquez sur votre projet "TP-RAGOps"
- Vous verrez la trace apparaître !

Ce que vous verrez :

```
┌─────────────────────────────────────────────────────────┐
│ RAG_Query
│ Durée : 4.2s
│
│ Input: "Quel est le dispositif ?"
│ Output: "Le dispositif est Cassation"
│
│ Timeline:
│ ████ ⋯⋯⋯⋯ Retrieval (0.3s)
│ ⋯⋯⋯⋯ ████████ Generation (3.9s)
└─────────────────────────────────────────────────────────┘
```

Cliquez sur une étape pour voir les détails :

- Paramètres exacts (query, n_results)
- Résultats intermédiaires

- Erreurs éventuelles

ÉTAPE 13 : Analyser et diagnostiquer

Questions à poser :

1. **Quelle étape est la plus lente ?**
    - Regardez la timeline dans LangSmith
    - Notez les durées
2. **Est-ce normal ?**
    - Retrieval : Doit être rapide (<500ms)
    - Generation : Peut être lent (1-3s)
3. **Que faire si c'est trop lent ?**

Si Retrieval est lent (>1s) :

- Indexer sur GPU
- Réduire la taille du corpus
- Optimiser les embeddings

Si Generation est lent (>5s) :

- Passer à `mistral-small` (plus rapide)
- Réduire le nombre de tokens générés
- Activer le streaming

Rapport (1 page)

```
# Rapport d'Évaluation

## 1. Scores Ragas
- context_precision : ___
- context_recall : ___
```

```
- faithfulness : ___
- answer_relevancy : ___

## 2. Diagnostic
Quelle est votre note la plus basse ?
Pourquoi ?

## 3. Actions
Que pourriez-vous améliorer ?
1. ...
2. ...
3. ...

## 4. Performance
Quelle étape est la plus lente ?
Que faire ?
```

CHECKLIST FINALE

vérifiez :

Partie 1 :

- 15 questions générées
- Fichier JSON valide

Partie 2 :

- Ragas fonctionne
- 4 scores calculés
- Diagnostic fait

Partie 3 :

- LLM Judge fonctionne
- Au moins 1 réponse notée

Partie 4 :

- Trace créée
- Étape lente identifiée